Project Organization and Student Work Description (suite): Week 3 Guide

4. WEEK 3 – Functional Decomposition & Architecture Definition

After completing the Week 1–2 deliverables, you will now move into **Week 3**, where you will transform customer-level functions into a structured and formal functional model. This model will serve as the foundation for your detailed system design and architectural development in the coming weeks.
This week focuses on applying the SADT (Structured Analysis and Design Technique) methodology to decompose and understand your BMS at a functional level.

4.1 Objectives:

By the end of this week, each team will:

1. Transform the requirements matrix into a functional architecture.
2. Apply SADT (IDEF0) methodology correctly.
3. Produce a validated A0 and A1–An decomposition.
4. Define all Inputs, Outputs, Controls, and Mechanisms (ICOM).
5. Clarify cross-team interfaces.

At the end of this week, your system must be functionally structured and ready for detailed design (Week 4).

4.2 Steps Workflow (Check Annex-Week 3)

Step 1 – Each team should draw the global Function (A0)

Step 2- Identify Your main sub-project Function (Ai)

Team	Ai Function
Team 1	Acquire Measurements
Team 2	Estimate Battery States
Team 3	Ensure Battery Safety
Team 4	Manage BMS Integration

Each team defines its **Ai function**.

Step 3 – Extract ICOM from Requirements Matrix

1. Use your RTM.
2. Ask:
   - Which requirement defines an input?
   - Which defines a constraint?
   - Which defines performance?
   - Which defines output?
3. Prepare ICOM table and draw Ai.
4. Verify:
   - Does every output come from an input?
   - Are constraints shown as controls?

Step 4 –Sub-level decomposition:

Decompose Ai function into Ai.1-Ai.n sub-functions, where i= 1,2,3,4 and n is the sub-levels decompositions.

Step 5 – Cross-Team Integration

After individual diagrams:

1. Check interactions between teams and identify interfaces connections.
2. Check interface consistance.
3. Verify no circular dependency without logic.

4.3 Deliverables

By the end of Week 3, each team must submit:

- Document including:

- A0 & A1-An Decompositions
- ICOM tables
- Cross-team interface control

- Requirement ↔ Block «satisfy» Mapping (update RTM)

Annex-Week 3

Understanding SADT/IDEF0

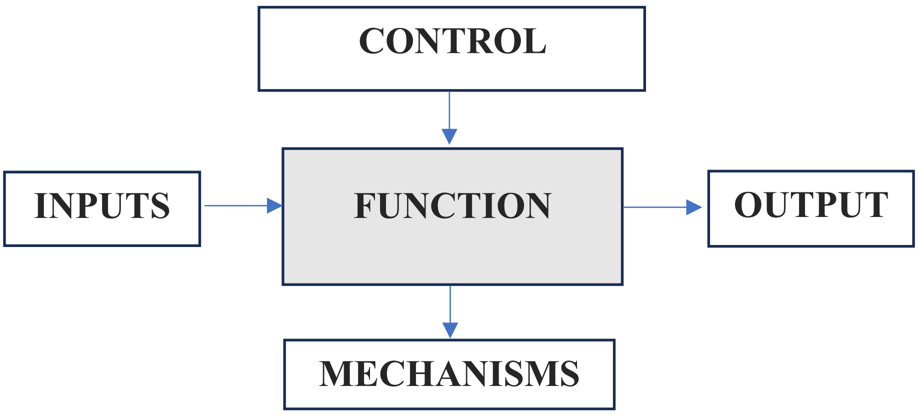
1. What is SADT?

SADT (**Structured Analysis and Design Technique**) is a methodology developed in the 1970s for analyzing and designing complex systems. It was later standardized as **IDEF0** (Integration Definition for Function Modeling).
Key Characteristics:
Function-Oriented: Focuses on WHAT the system does (verbs), not HOW it does it.
Top-Down Decomposition: Breaks complex functions into simpler subfunctions hierarchically.
Standardized Notation: Uses boxes and arrows with specific meanings.
ICOM Model: Every function has Inputs, Controls, Outputs, and Mechanisms.

2. The ICOM Model

Every SADT function box is characterized by four types of arrows representing the ICOM elements:

Element	**Definition**	**Position**
Input (I)	Data, material, or energy that is transformed or consumed by the function	Left side
Control (C)	Conditions, constraints, rules, or standards that govern how the function operates	Top
Output (O)	The result or product of the function	Right side
Mechanism (M)	Resources (people, hardware, software) that perform the function	Bottom



3. Specific ICOM Examples-BMS

For a Battery Management System, here are typical examples of each ICOM element:

Input (I)	**Control (C)**	**Output (O)**	**Mechanism (M)**
Cell voltage measurements CAN messages from vehicle controller …..	ISO 26262 safety standard SOC estimation algorithm parameters ……	Contactor control signals SOC/SOH estimates …..	MCU CAN transceiver Balancing circuit …….

4. Creating the A0 Diagram

4.1. What is A0?

The A0 diagram represents the **highest level of abstraction** of your system. It shows the entire BMS as a single function box with all its external interfaces.
Key Principles:
One Box Only: A0 contains exactly one function box
System Boundary: Shows what enters and leaves your BMS
Verb-Noun Title: Use active verbs (e.g., "Manage Battery System" not "Battery Management")
External Perspective: Think from outside the system looking in

4.2. Step-by-Step A0 Creation

- Step 1: Define the Main Function
Choose a clear, concise name for your BMS's primary function. Example: "Manage Battery Pack"
- Step 2: Identify All Inputs
What physical or informational elements enter your BMS?
Example: Individual cell voltages (from each cell in series)
- Step 3: Identify All Controls
What rules, standards, or parameters govern your BMS operation?
Example: ISO 26262 (Automotive Functional Safety Standard)
Step 4: Identify All Outputs
What does your BMS produce or send out?
Example: Main contactor control signals (Pre-charge, Positive, Negative)
Step 5: Identify All Mechanisms
What resources enable your BMS to function?
Temperature sensors (NTC thermistors)

4.3. A0 Diagram Example

Below is a textual representation of what your A0 diagram should include. **You will create the actual graphical diagram using drawing tools** (Visio, Draw.io, PowerPoint, etc.).

```
┌──────────────────────────────────────────────────────────────────┐
│                                                                  │
│                    CONTROLS (Top):                               │
│                    • ISO 26262                                   │
│                    • ..........                                  │
│                    • ..........                                  │
│                    • ..........                                  │
│                              ↓                                   │
│                                                                  │
│  INPUTS (Left):       ┌────────────────┐    OUTPUTS (Right):     │
│  • Cell Voltages →    │                │  → • Contactor Commands │
│  • .......... .. →    │  MANAGE        │  → • ..........         │
│  • ...........   →    │  BATTERY       │  → • ..........         │
│  • ............  →    │  PACK (A0)     │  → • ..........         │
│                       │                │  → • ..........         │
│                       └────────────────┘                         │
│                              ↑                                   │
│                    MECHANISMS (Bottom):                          │
│                    • .......... Microcontroller                  │
│                    • ..........                                  │
│                    • ..........                                  │
│                    • ..........                                  │
│                    • ........                                    │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

5. Creating the Ai Diagram (Decomposition)

The Ai diagram **decomposes** the A0 function into its major subfunctions. It shows how the top-level function is accomplished by breaking it into n lower-level activities.
Key Principles:
Number of Boxes: not too many, not too few ≥3.
Sequential or Parallel: Can show sequence (data flows from one box to next) or parallel activities.
Internal Flows: Arrows between boxes show internal data/control flow.
Same ICOM: External inputs/outputs match the A0 diagram (boundary consistency).

6. Creating the Ai.n Level Diagram (Decomposition)

The Ai.n level represents the next layer of decomposition where you break down each Ai subfunction into even more detailed sub-subfunctions. While Ai shows the main sub-projects functional blocks, Ai.n shows how each of those blocks actually accomplishes its work.

Hierarchical Structure:

```
A0 (Global Function)
└── A1 Main sub-project Function
      └── A1.1 (Detailed Sub-functions: ≥3 boxes per A1 function)
            └── A1.1.1 (Even more detail if needed)
```

Example: Sub-level Hierarchy

```
A0: Manage Battery Pack
 |
 ├── A1.1: Acquire Measurements ← We decompose this
 |     |
 |     ├── A1.1.1: Measure Cell Voltages
 |     |     └── (Could go to A3 if needed: configure AFE, read SPI, convert...)
 |     |
```